

Федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники
Направление подготовки 09.03.04 Программная инженерия
Дисциплина «Технологии виртуализации»

Отчёт
по лабораторной работе №3.1
«Знакомство с Docker»

Выполнил:
Стрельбицкий Илья Павлович Р3413
Проверил:
Белозубов Александр Владимирович

Санкт-Петербург

2025 г.

Содержание

Введение.....	3-4
Выполнение	5-28
Заключение	29

Введение

Настоящая лабораторная работа посвящена освоению практических навыков работы с платформой контейнеризации Docker. В современной разработке и эксплуатации ПО (DevOps) Docker является ключевым инструментом, позволяющим инкапсулировать приложение со всеми его зависимостями в универсальную единицу развертывания — контейнер. Это обеспечивает переносимость, воспроизводимость и изоляцию сред, что критически важно для создания надежных и масштабируемых систем. В отличие от традиционной виртуализации, контейнеры, работая на уровне операционной системы, обеспечивают минимальные накладные расходы и высокую скорость запуска.

Практическая часть выполнялась в среде Ubuntu. В ходе работы был пройден полный цикл взаимодействия с экосистемой Docker: от установки и базового управления до сборки собственных образов и оркестрации сложных многоконтейнерных приложений с помощью Docker Compose. В качестве учебных стендов были развернуты и настроены реальные сервисы: веб-сервер Nginx, СУБД MariaDB, облачное хранилище NextCloud и веб-интерфейс для управления виртуальными машинами phpVirtualBox.

Особое внимание было уделено интеграции в процессы непрерывной интеграции и доставки (CI/CD) через публикацию собранного образа в центральном реестре Docker Hub. Завершающим этапом стала демонстрация универсальности платформы путем запуска готового контейнеризированного приложения из публичного репозитория.

Цель работы

Приобрести практические навыки работы с Docker, изучив ключевые аспекты жизненного цикла контейнеров: от установки и настройки среды до создания, управления и публикации образов. Работа включает освоение команд CLI, создание собственных Dockerfile, оркестрацию нескольких сервисов и интеграцию с облачным реестром.

Задачи работы

1. Развернуть платформу Docker в ОС Ubuntu, настроив корректные права доступа для работы без привилегий суперпользователя.
2. Проверить корректность установки, изучить и отработать базовые команды интерфейса командной строки (CLI) для управления Docker.
3. Разработать Dockerfile для сборки пользовательского образа веб-сервера с HTML-страницей и обеспечить его запуск на заданном порту (8080).
4. Запустить контейнер с СУБД MariaDB, выполнить подключение к ней и создать тестовую базу данных, имя которой соответствует фамилии студента.
5. Развернуть контейнер с облачным хранилищем NextCloud, организовав постоянное хранение данных на хосте и зарегистрировав тестовых пользователей.
6. Установить и запустить контейнер с веб-интерфейсом для управления виртуализацией — phpVirtualBox.
7. Реализовать комплексное многоконтейнерное приложение с использованием Docker Compose, объединяющее сервисы NextCloud, веб-сервер и базу данных, и обеспечить доступ к нему через порт 2022.
8. Зарегистрироваться на платформе Docker Hub, создать репозиторий и опубликовать в нем собранный ранее пользовательский образ.
9. Продемонстрировать простоту развертывания сторонних решений путем запуска готового контейнера (например, веб-игры 2048) из публичного реестра Docker Hub.
10. Систематизировать полученный опыт, составив справочное описание основных команд Docker для работы с образами и контейнерами.

Выполнение

1. Установка Docker

Установлен пакет docker в системе ubuntu. Пользователь добавлен в группу docker для выполнения команд без sudo. Демон Docker включен в автозагрузку, Virtualbox тоже был установлен.

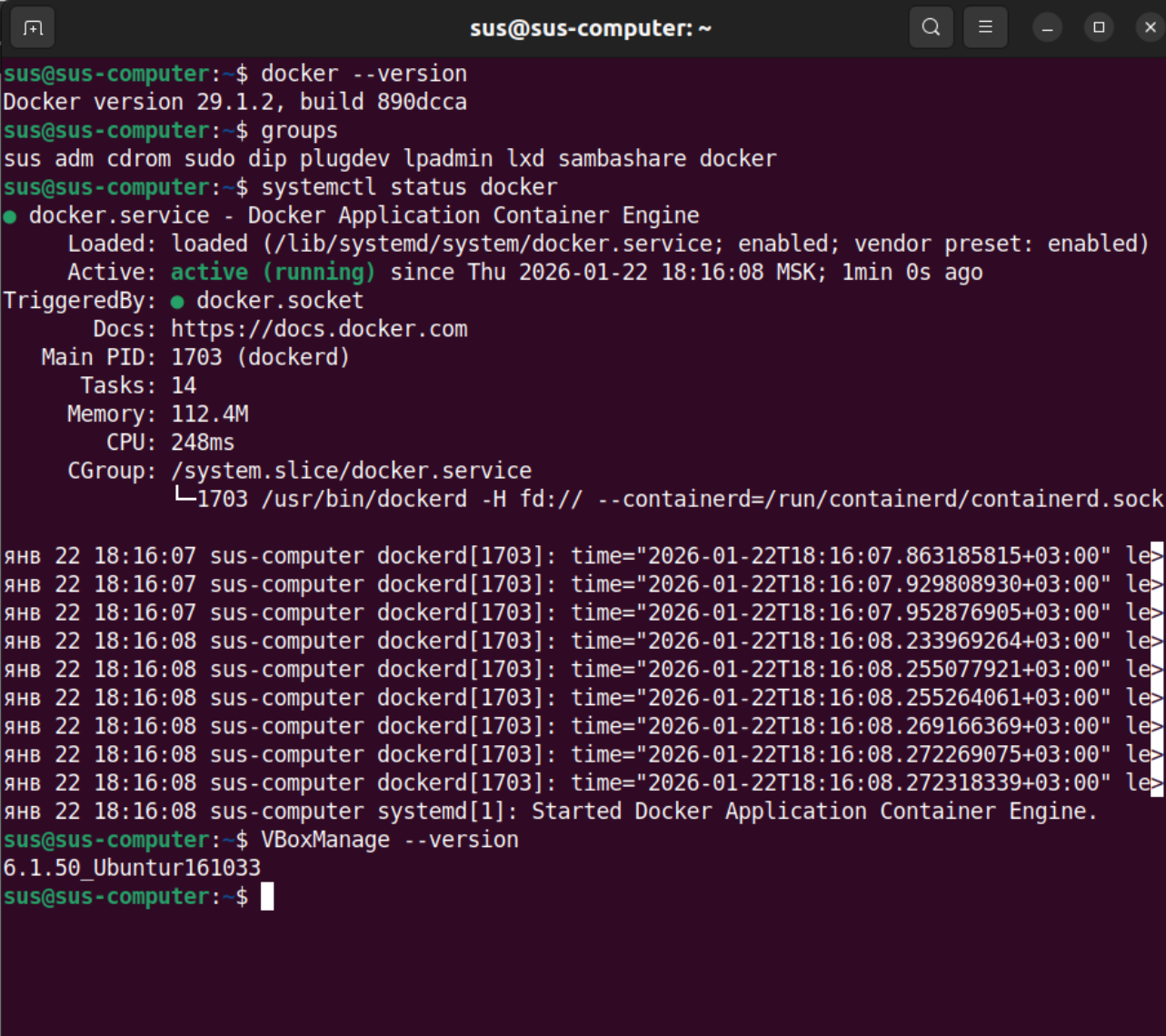
A terminal window titled 'sus@sus-computer: ~' with standard Ubuntu window controls. The terminal shows the following commands and output:
1. `docker --version` outputs: `Docker version 29.1.2, build 890dcca`
2. `groups` outputs: `sus adm cdrom sudo dip plugdev lpadmin lxd sambashare docker`
3. `systemctl status docker` outputs detailed service status for `docker.service`, showing it is `active (running)` since `Thu 2026-01-22 18:16:08 MSK`. It lists various metrics like `Main PID: 1703 (dockerd)`, `Memory: 112.4M`, and `CPU: 248ms`.
4. A series of log entries for `dockerd` and `systemd` are shown, indicating the engine has started successfully.
5. `VBoxManage --version` outputs: `6.1.50_Ubuntur161033`
The prompt returns to `sus@sus-computer:~$`.

Рисунок 1. Установлены Docker и VirtualBox.

2. Запуск тестового контейнера

Пробуем запустить тестовый контейнер `hello-strelbitskiy`, как показано на рисунке 2:

```
sus@sus-computer:~$ docker run hello-strelbitskiy
Unable to find image 'hello-strelbitskiy:latest' locally
docker: Error response from daemon: pull access denied for hello-strelbitskiy, repository
does not exist or may require 'docker login': denied: requested access to the resource is
denied

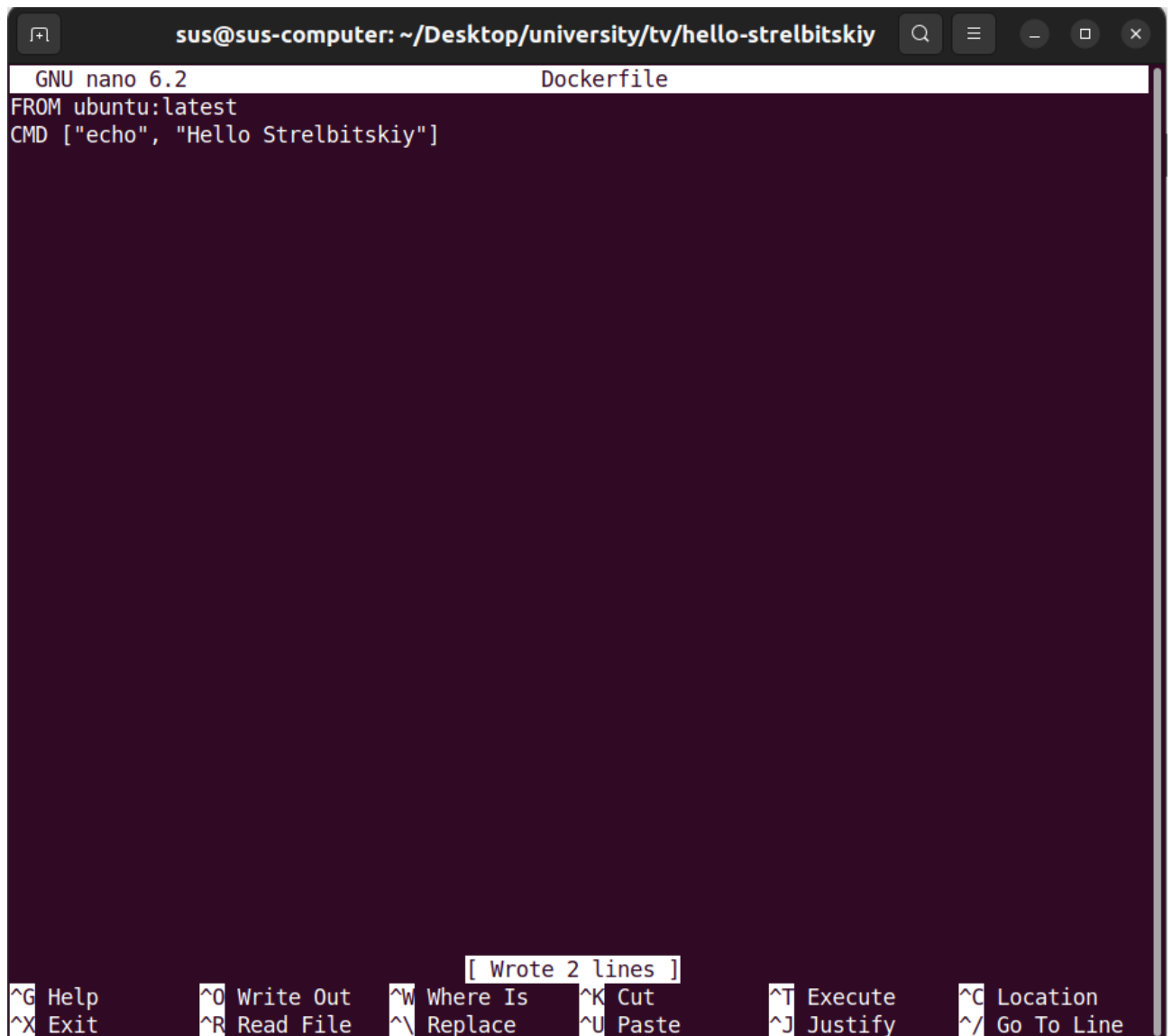
Run 'docker run --help' for more information
sus@sus-computer:~$
```

Рисунок 2. Неудачная попытка запустить тестовый контейнер hello-strelbitskiy

Получаем ошибку, поскольку образа hello-strelbitskiy не существует. Исправим это, создав образ hello-strelbitskiy — для этого создадим Dockerfile как показано на рисунках 3 и 4.

```
sus@sus-computer: ~/Desktop/university/tv/hello-strelbitskiy
sus@sus-computer:~/Desktop/university/tv$ mkdir hello-strelbitskiy
sus@sus-computer:~/Desktop/university/tv$ cd hello-strelbitskiy/
sus@sus-computer:~/Desktop/university/tv/hello-strelbitskiy$ nano Dockerfile
sus@sus-computer:~/Desktop/university/tv/hello-strelbitskiy$
```

Рисунок 3. Создание Dockerfile для образа



The image shows a terminal window with a dark background. At the top, the window title is "sus@sus-computer: ~/Desktop/university/tv/hello-strelbitskiy". Below the title bar, the nano editor interface is visible. The top status bar shows "GNU nano 6.2" and "Dockerfile". The main editing area contains two lines of text: "FROM ubuntu:latest" and "CMD ["echo", "Hello Strelbitskiy"]". At the bottom of the editor, a status bar displays "[Wrote 2 lines]" and a series of keyboard shortcuts: ^G Help, ^O Write Out, ^W Where Is, ^K Cut, ^T Execute, ^C Location, ^X Exit, ^R Read File, ^\ Replace, ^U Paste, ^J Justify, and ^_ Go To Line.

```
sus@sus-computer: ~/Desktop/university/tv/hello-strelbitskiy
GNU nano 6.2 Dockerfile
FROM ubuntu:latest
CMD ["echo", "Hello Strelbitskiy"]

[ Wrote 2 lines ]
^G Help      ^O Write Out ^W Where Is  ^K Cut      ^T Execute  ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify  ^_ Go To Line
```

Рисунок 4. Содержимое Dockerfile

Создаем контейнер и запускаем его для проверки работоспособности. Действия показаны на рисунке 5.

```

sus@sus-computer: ~/Desktop/university/tv/hello-strelbitskiy$ docker build -t hello-strelbitskiy .
[+] Building 13.5s (5/5) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile              0.1s
=> => transferring dockerfile: 91B                               0.0s
=> [internal] load metadata for docker.io/library/ubuntu:latest 2.7s
=> [internal] load .dockerignore                                0.0s
=> => transferring context: 2B                                    0.0s
=> [1/1] FROM docker.io/library/ubuntu:latest@sha256:cd1dba651b3080c3686ecf4e3c42 10.3s
=> => resolve docker.io/library/ubuntu:latest@sha256:cd1dba651b3080c3686ecf4e3c422 0.0s
=> => sha256:cd1dba651b3080c3686ecf4e3c4220f026b521fb76978881737d2 6.69kB / 6.69kB 0.0s
=> => sha256:a4453623f2f8319cfff65c43da9be80fe83b1a7ce689579b475867d69 424B / 424B 0.0s
=> => sha256:493218ed0f404132311952996fea8ce85e50c49f5a717f26f25c5 2.30kB / 2.30kB 0.0s
=> => sha256:a3629ac5b9f4680dc2032439ff2354e73b06aecc2e68f0035a2 29.73MB / 29.73MB 9.6s
=> => extracting sha256:a3629ac5b9f4680dc2032439ff2354e73b06aecc2e68f0035a2d7c001c 0.4s
=> exporting to image                                           0.1s
=> => exporting layers                                           0.0s
=> => writing image sha256:526d832ca294bbce79a9ca8cffe600a5be0dd7aa3589f7e720b36d 0.0s
=> => naming to docker.io/library/hello-strelbitskiy            0.0s
sus@sus-computer: ~/Desktop/university/tv/hello-strelbitskiy$ docker run hello-strelbitskiy
Hello Strelbitskiy
sus@sus-computer: ~/Desktop/university/tv/hello-strelbitskiy$

```

Рисунок 5. Создание контейнера и проверка его работоспособности.

3. Контейнер с Nginx и пользовательской страницей

Создаем каталог /home/www/html, в нем же создаем файл index.html, также создаем файл Dockerfile на базе образа nginx:latest, который пробрасывает порт 8080 и копирует страницу — рисунок 7.


```

sus@sus-computer:~/Desktop/university/tv/src$ cat /home/www/html/index.html
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <title>Технологии виртуализации ЛРЗ</title>
</head>
<body>
  <h1>Стрельбицкий Илья Павлович</h1>
  <p>Студент группы Р3413</p>
</body>
</html>
sus@sus-computer:~/Desktop/university/tv/src$ cat nginx.Dockerfile
FROM nginx:latest
LABEL maintainer="Стрельбицкий Илья Павлович"
WORKDIR /usr/share/nginx/html
COPY index.html .
EXPOSE 8080
CMD ["nginx", "-g", "daemon off;"]
sus@sus-computer:~/Desktop/university/tv/src$

```

Рисунок 7. Содержимое html страницы и Dockerfile

Собираем образ и запускаем его с монтированием тома — рисунок 8.

```

sus@sus-computer:~/Desktop/university/tv/src$ docker build -t my-nginx -f nginx.Dockerfile
[+] Building 2.9s (8/8) FINISHED                                docker:default
=> [internal] load build definition from nginx.Dockerfile      0.0s
=> => transferring dockerfile: 228B                             0.0s
=> [internal] load metadata for docker.io/library/nginx:latest 2.7s
=> [internal] load .dockerignore                               0.0s
=> => transferring context: 2B                                    0.0s
=> [1/3] FROM docker.io/library/nginx:latest@sha256:c881927c4077710ac4b1da63b83aa1 0.0s
=> [internal] load build context                               0.0s
=> => transferring context: 32B                                    0.0s
=> CACHED [2/3] WORKDIR /usr/share/nginx/html                  0.0s
=> CACHED [3/3] COPY index.html .                              0.0s
=> exporting to image                                           0.0s
=> => exporting layers                                           0.0s
=> => writing image sha256:49ca7222d6cbe95dba70a3936ac7ac4fb67a1ecdb0cc7c6a487c7c3 0.0s
=> => naming to docker.io/library/my-nginx                     0.0s
sus@sus-computer:~/Desktop/university/tv/src$ docker run -d -p 8080:80 -v /home/www/html:/usr/share/nginx/html:ro --name my-webserver my-nginx
45f1171a2bc9b8f4af21b9f7ac50855ffe48731eb9f2bffc07cf757714d3b8f7
sus@sus-computer:~/Desktop/university/tv/src$

```

Рисунок 8. Сборка и запуск образа

Чтобы убедиться что образ действительно запущен и работает, открываем браузер по адресу <http://localhost:8080> — открываемая страница показана на рисунке 9.



Стрельбицкий Илья Павлович

Студент группы Р3413

Рисунок 9. Открываемая страница

4. Контейнер с MariaDB

Запускаем контейнер mariadb с монтированием тома /home/DB с пробрасыванием порта 3306 — рисунок 10.

```
sus@sus-computer:~/Desktop/university/tv/src$ docker run -d --name my-mariadb \
> -v /home/DB:/var/lib/mysql \
> -e MYSQL_ROOT_PASSWORD=admin \
> -p 3306:3306 \
> mariadb:12.1.1-rc
Unable to find image 'mariadb:12.1.1-rc' locally
12.1.1-rc: Pulling from library/mariadb
20043066d3d5: Pull complete
80bde5efc112: Pull complete
8e58c403911e: Pull complete
72fdec3d31db: Pull complete
9caa2f8cb036: Pull complete
48fe8929c4b2: Pull complete
4836bc127f81: Pull complete
f00d5a3f2905: Pull complete
Digest: sha256:f18fab8692186f1a553ffd859728734ec91119d21cfa39cdf1b2f59265c972de
Status: Downloaded newer image for mariadb:12.1.1-rc
3e3bde6951b2094c15046f06a30d57ff662e581c82b5d3256eb3402ac0e49d44
sus@sus-computer:~/Desktop/university/tv/src$
```

Рисунок 10. Запуск контейнера mariadb

При помощи docker exec выполняем подключение к СУБД и создаем базу данных с именем strelbitskiy — рисунок 11.

```

sus@sus-computer:~/Desktop/university/tv/src$ docker exec -it my-mariadb mariadb -u root -
padmin -e "CREATE DATABASE strelbitskiy; SHOW DATABASES;"
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| strelbitskiy |
| sys |
+-----+
sus@sus-computer:~/Desktop/university/tv/src$

```

Рисунок 11. Создание базы данных в MariaDB

Поскольку был монтирован том /home/DB, то в этой директории сохранилась созданная база данных — рисунок 12.

```

sus@sus-computer:~/Desktop/university/tv/src$ ls -la /home/DB
total 158492
drwxr-xr-x 6 999 docker      4096 янв 22 20:22 .
drwxr-xr-x 6 root root      4096 янв 22 20:15 ..
-rw-rw---- 1 999 docker 4907008 янв 22 20:15 aria_log.00000001
-rw-rw---- 1 999 docker    52 янв 22 20:15 aria_log_control
-rw-rw---- 1 999 docker    9 янв 22 20:15 ddl_recovery.log
-rw-rw---- 1 999 docker   706 янв 22 20:15 ib_buffer_pool
-rw-rw---- 1 999 docker 12582912 янв 22 20:15 ibdata1
-rw-rw---- 1 999 docker 100663296 янв 22 20:15 ib_logfile0
-rw-rw---- 1 999 docker 12582912 янв 22 20:15 ibtmp1
-rw-r--r-- 1 999 docker    14 янв 22 20:15 mariadb_upgrade_info
-rw-rw---- 1 999 docker    0 янв 22 20:15 multi-master.info
-rw-rw---- 1 999 docker   118 янв 22 20:15 .my-healthcheck.cnf
drwx----- 2 999 docker   4096 янв 22 20:15 mysql
drwx----- 2 999 docker   4096 янв 22 20:15 performance_schema
drwx----- 2 999 docker   4096 янв 22 20:22 strelbitskiy
drwx----- 2 999 docker  12288 янв 22 20:15 sys
-rw-rw---- 1 999 docker   24576 янв 22 20:15 tc.log
-rw-rw---- 1 999 docker 10485760 янв 22 20:15 undo001
-rw-rw---- 1 999 docker 10485760 янв 22 20:15 undo002
-rw-rw---- 1 999 docker 10485760 янв 22 20:15 undo003
sus@sus-computer:~/Desktop/university/tv/src$

```

Рисунок 12. Содержимое директории /home/DB

5. Контейнер с NextCloud

Запускаем официальный образ nexstcloud с пробрасыванием порта 8088 и монтированием каталога home/www/NextCloud – рисунок 13.

```
sus@sus-computer:~/Desktop/university/tv/src$ docker run -d --name my-nextcloud \
> -v /home/www/NextCloud:/var/www/html \
> -p 8088:80 nextcloud:latest
Unable to find image 'nextcloud:latest' locally
latest: Pulling from library/nextcloud
119d43eec815: Already exists
f888ecc1552d: Pull complete
1e82d9b5f876: Pull complete
d851cc61e688: Pull complete
f40e018d0724: Pull complete
ddc1c2038b99: Pull complete
9728c0f605d8: Pull complete
a192efa6c713: Pull complete
9c3d28e85308: Pull complete
0c8190b486bc: Pull complete
fd8d2946b911: Pull complete
9f7c018cbf82: Pull complete
a7ac8f681c14: Pull complete
6bdb3709272f: Pull complete
4f4fb700ef54: Pull complete
f43c61e60e4d: Pull complete
8fd37df2e0f2: Pull complete
9db15f85b501: Pull complete
b86b5b1fe6aa: Pull complete
691b53a17512: Pull complete
e5ceab09813f: Pull complete
5a87dc9c2eff: Pull complete
3269bfe7aea7: Pull complete
Digest: sha256:11a3a4f63bad8813c7455b4a3c473ccd1c41e2c48f55dec51718f15691e7568
Status: Downloaded newer image for nextcloud:latest
98c0afb7ece8fb0c9a57e81b0124643876959120af6d0a9c5d4d240e5da960b5
sus@sus-computer:~/Desktop/university/tv/src$
```

Рисунок 13. Создание контейнера с NextCloud

Логин и пароль от NextCloud – admin:admin – рисунок 14.

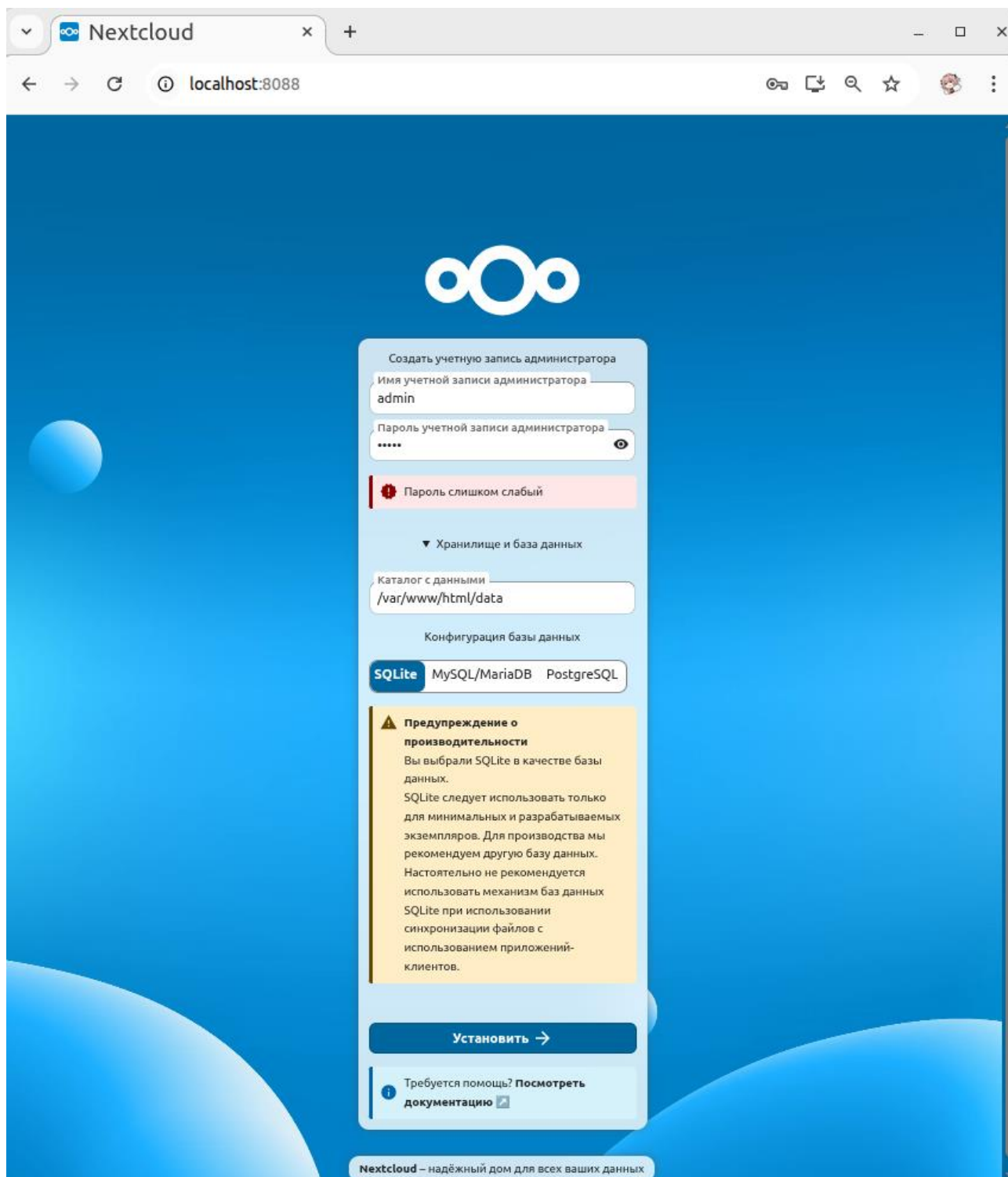


Рисунок 14. Настройка NextCloud

Через веб-интерфейс зарегистрированы два пользователя: студент (Стрельбицкий И.П.) и преподаватель (Белозубов А.В.) – рисунок 15, 16 и 17.

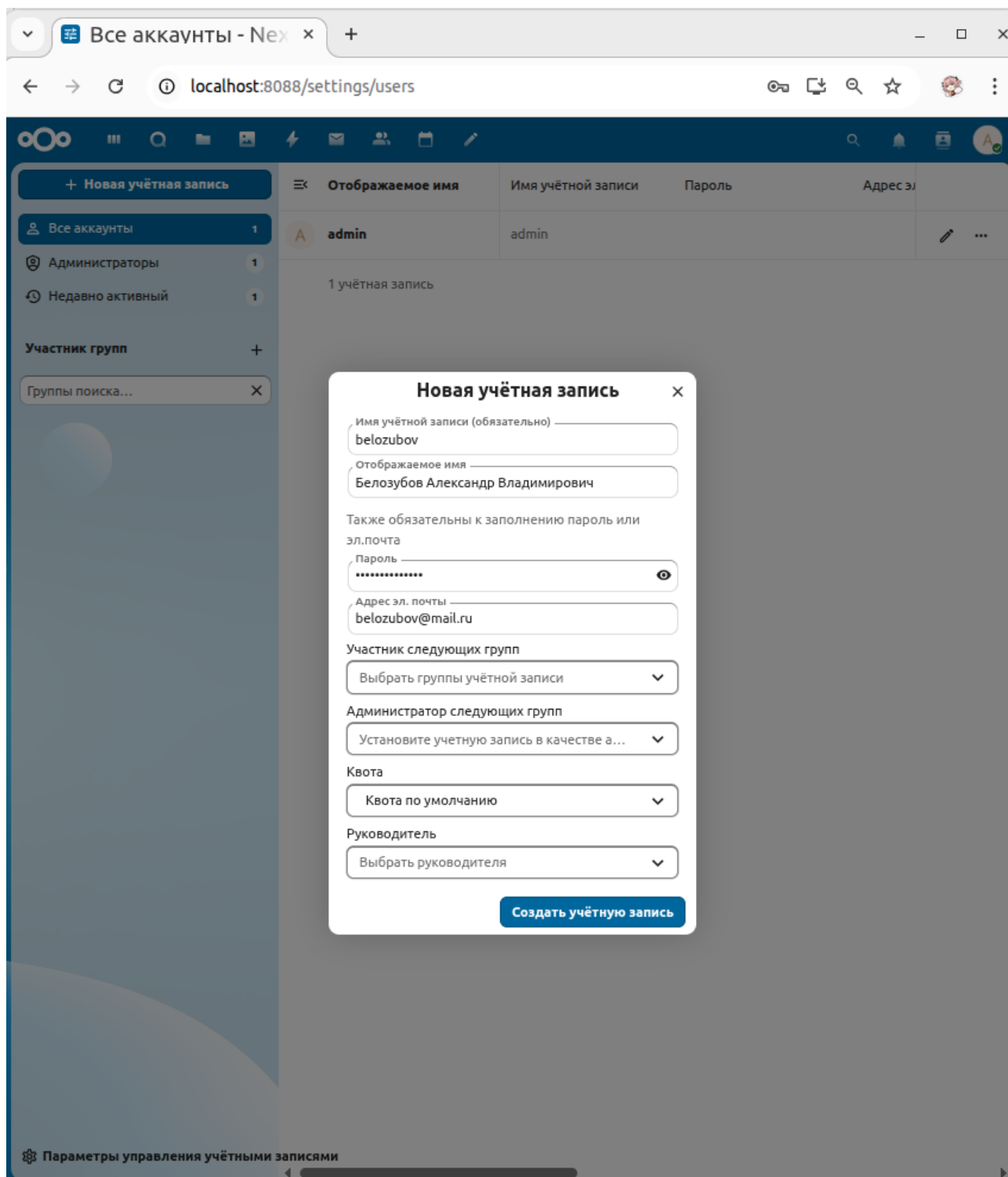


Рисунок 15. Создание учетной записи преподавателя

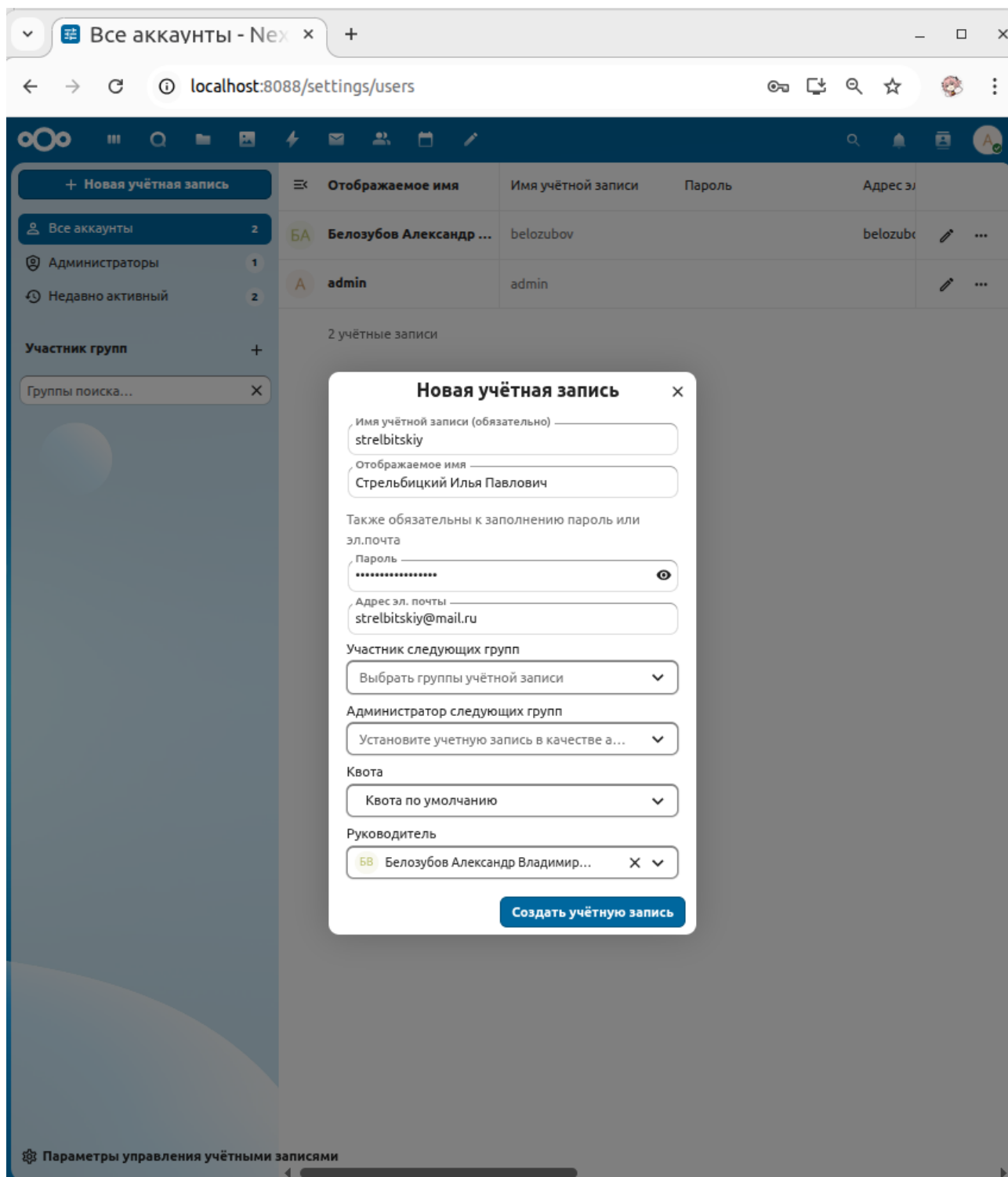


Рисунок 16. Создание учетной записи студента

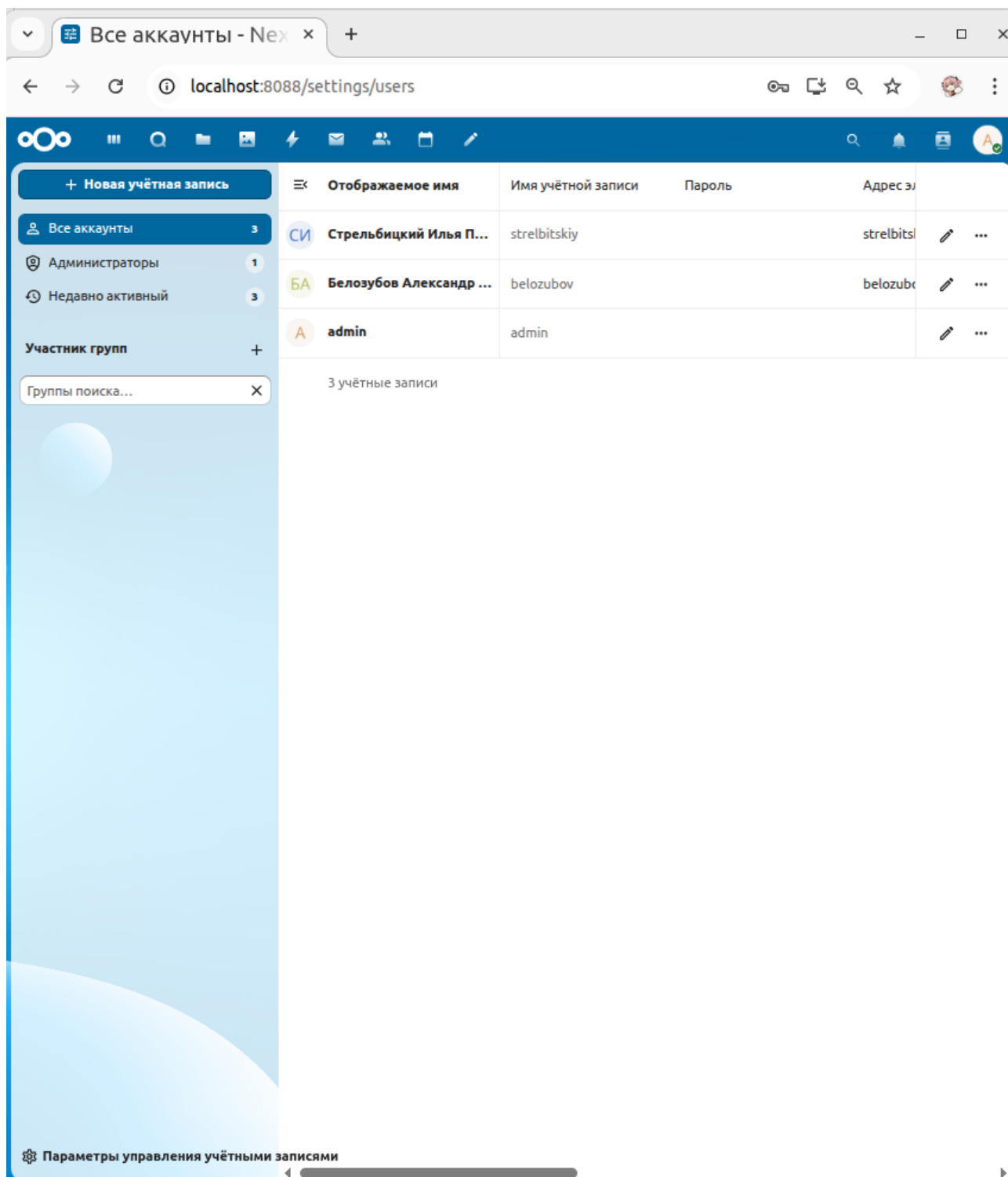


Рисунок 17. Раздел пользователи в NextCloud

6. Контейнер с phrvirtualbox

Запускаем vboxwebsrv — рисунок 18.

```
sus@sus-computer:~/Desktop/university/tv/src$ vboxwebsrv -H 0.0.0.0 -A null -b
Oracle VM VirtualBox web service Version 6.1.50_Ubuntu
(C) 2007-2024 Oracle Corporation
All rights reserved.
00:00:00.000789 main      VirtualBox web service 6.1.50_Ubuntu r161033 linux.amd64 (Aug  2
2024 10:31:49) release log
00:00:00.000791 main      Log opened 2026-01-22T18:00:18.950541000Z
00:00:00.000792 main      Build Type: release
00:00:00.000794 main      OS Product: Linux
00:00:00.000795 main      OS Release: 6.8.0-90-generic
00:00:00.000795 main      OS Version: #91~22.04.1-Ubuntu SMP PREEMPT_DYNAMIC Thu Nov 20 15:
20:45 UTC 2
00:00:00.000824 main      DMI Product Name: Vostro 3400
00:00:00.000828 main      DMI Product Version:
00:00:00.000833 main      Firmware type: UEFI
00:00:00.001340 main      Secure Boot: Disabled
00:00:00.001368 main      Host RAM: 15720MB (15.3GB) total, 12255MB (11.9GB) available
00:00:00.001370 main      Executable: /usr/lib/virtualbox/vboxwebsrv
00:00:00.001370 main      Process ID: 19446
00:00:00.001370 main      Package type: LINUX_64BITS_GENERIC (OSE)
sus@sus-computer:~/Desktop/university/tv/src$
```

Рисунок 18. Запуск веб-сервиса VirtualBox

Запускаем образ phpVirtualBox — рисунок 19.

```
sus@sus-computer:~/Desktop/university/tv/src$ docker run -d \
--name phpvbox \
-p 8081:80 \
-e ID_HOSTPORT=172.17.0.1:18083 \
-e ID_USER='' \
-e ID_PW='' \
-e ID_NAME="My VirtualBox" \
--restart unless-stopped \
jazzdd/phpvirtualbox
Unable to find image 'jazzdd/phpvirtualbox:latest' locally
latest: Pulling from jazzdd/phpvirtualbox
63b65145d645: Pull complete
82bcb2b43f07: Pull complete
5900127473cc: Pull complete
0a5b4ded0d70: Pull complete
a4a4bd376941: Pull complete
Digest: sha256:a881ae25a2e6349484b9c8acada28d81c07e86356ec660d6c9ba7142ef86d040
Status: Downloaded newer image for jazzdd/phpvirtualbox:latest
8e397059c365af0bc61b80138e41b885580cf4806d212e650b3572c7c7d2982a
sus@sus-computer:~/Desktop/university/tv/src$
```

Рисунок 19. Запуск образа phpVirtualBox

Логин, пароль от phpVirtualBox — admin:admin — рисунок 20.

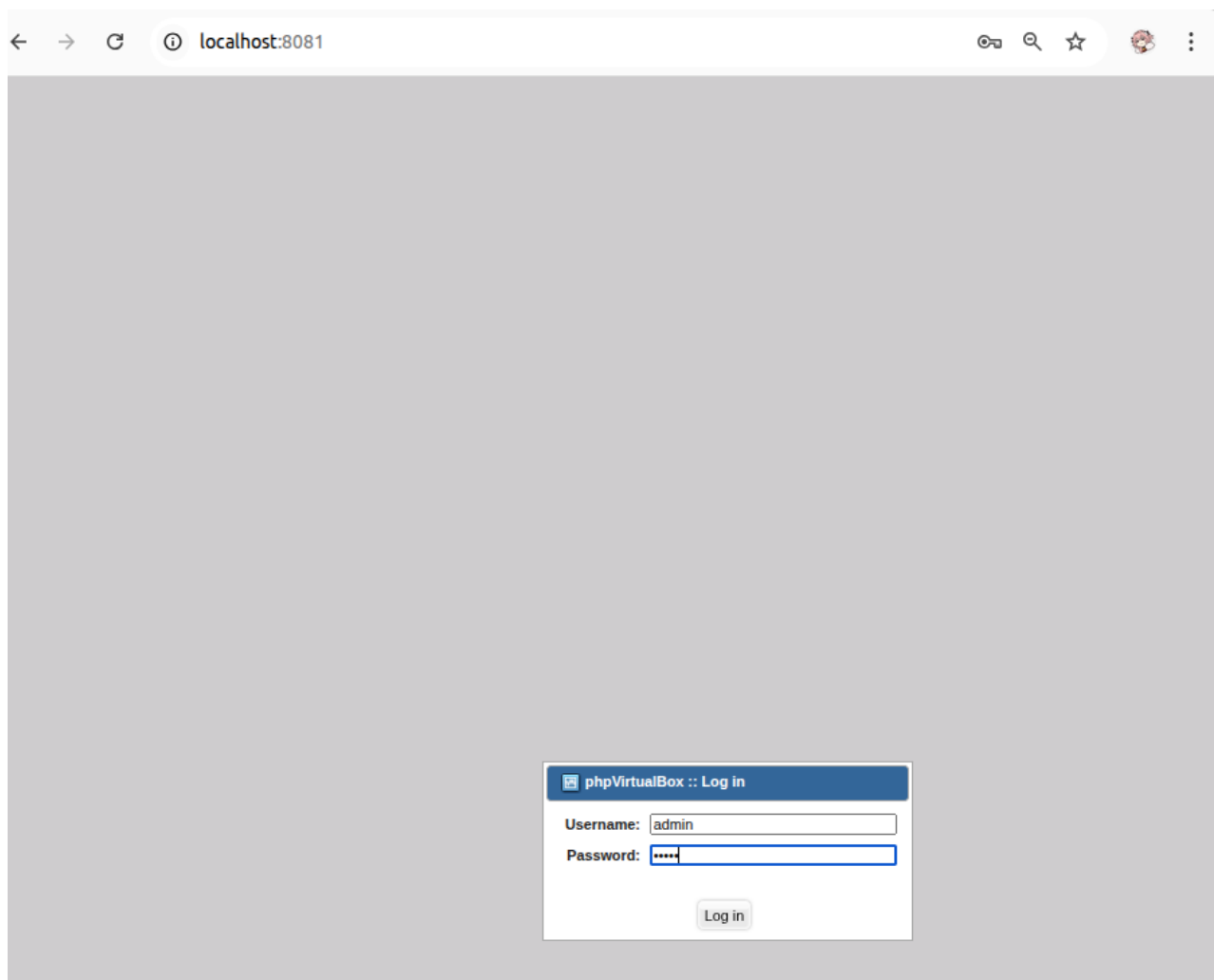


Рисунок 20. Вход в phpVirtualBox через веб-интерфейс

Открываем интерфейс phpVirtualBox для проверки работы — рисунок 21.

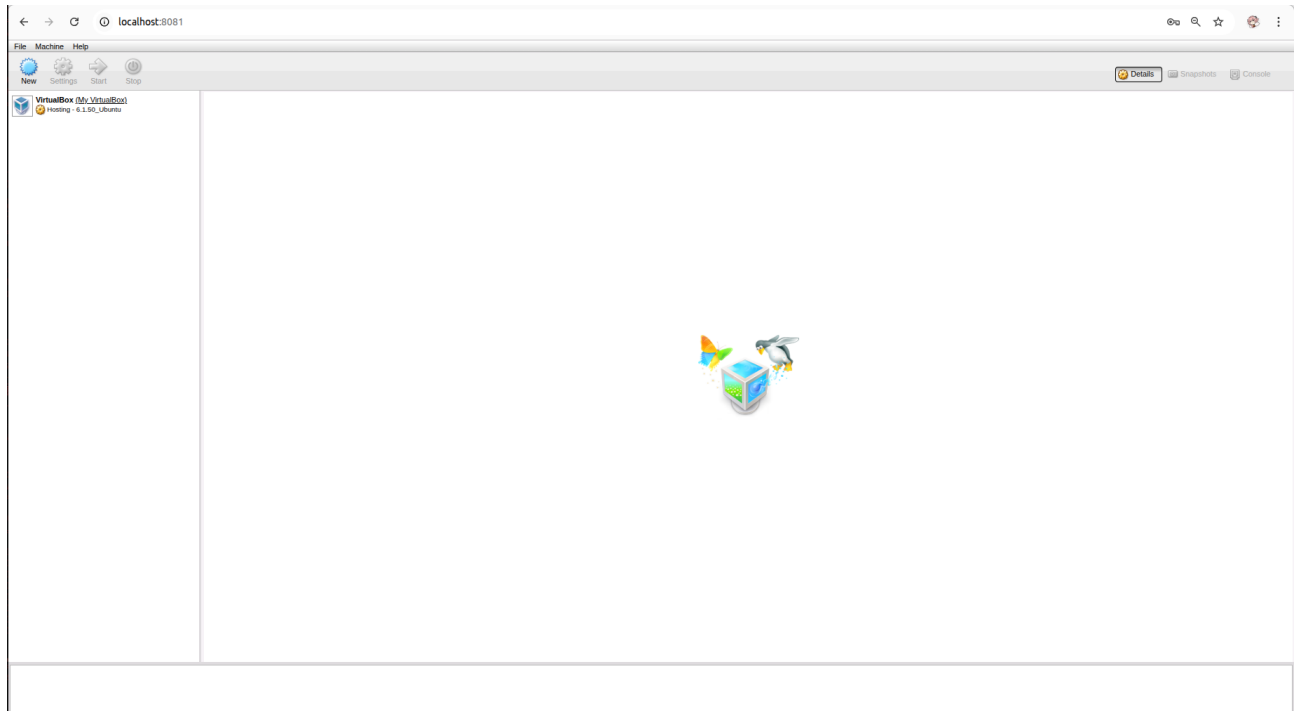


Рисунок 21. Веб-интерфейс phpVirtualBox

7. Docker Compose

Создаем файл `docker-compose.yml`, который описывает 3 сервиса: `db` (MariaDB), `app` (NextCloud), `web` (Nginx) — рисунок 22.

```
sus@sus-computer:~/Desktop/university/tv/src$ cat docker-compose.yml
services:
  web:
    build:
      context: .
      dockerfile: nginx.Dockerfile
    container_name: nginx-webserver
    restart: unless-stopped
    ports:
      - "8080:80"
    volumes:
      - /home/www/html:/usr/share/nginx/html:ro
    networks:
      - nextcloud-network

  db:
    image: mariadb:12.1.1-rc
    container_name: nextcloud-mariadb
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD: admin
      MYSQL_DATABASE: mariadb
    volumes:
      - /home/DB:/var/lib/mysql
    networks:
      - nextcloud-network

  nextcloud:
    image: nextcloud:latest
    container_name: nextcloud-app
    restart: unless-stopped
    ports:
      - "2022:80"
    environment:
      MYSQL_HOST: db
      MYSQL_DATABASE: mariadb
      MYSQL_USER: root
      MYSQL_PASSWORD: admin
    volumes:
      - /home/www/NextCloud:/var/www/html
    depends_on:
      - db
      - web
    networks:
      - nextcloud-network

networks:
  nextcloud-network:
    driver: bridge
sus@sus-computer:~/Desktop/university/tv/src$
```

Рисунок 22. Содержимое docker-compose.yml

Запускаем сервисы и смотрим их состояние при помощи команды `docker-compose ps` — рисунок 23.

```
sus@sus-computer:~/Desktop/university/tv/src$ docker-compose up -d
Starting nginx-webserver ... done
Starting nextcloud-mariadb ... done
Creating nextcloud-app ... done
sus@sus-computer:~/Desktop/university/tv/src$ docker-compose ps
```

Name	Command	State	Ports
nextcloud-app	/entrypoint.sh apache2-for ...	Up	0.0.0.0:2022->80/tcp,:::2022->80/tcp
nextcloud-mariadb	docker-entrypoint.sh mariadb	Up	3306/tcp
nginx-webserver	/docker-entrypoint.sh nginx	Up	0.0.0.0:8080->80/tcp,:::8080->80/tcp, 8080/tcp

```
sus@sus-computer:~/Desktop/university/tv/src$
```

Рисунок 23. Запуск сервисов

Открываем страницу NextCloud по адресу <http://localhost:2022> — рисунок 24.

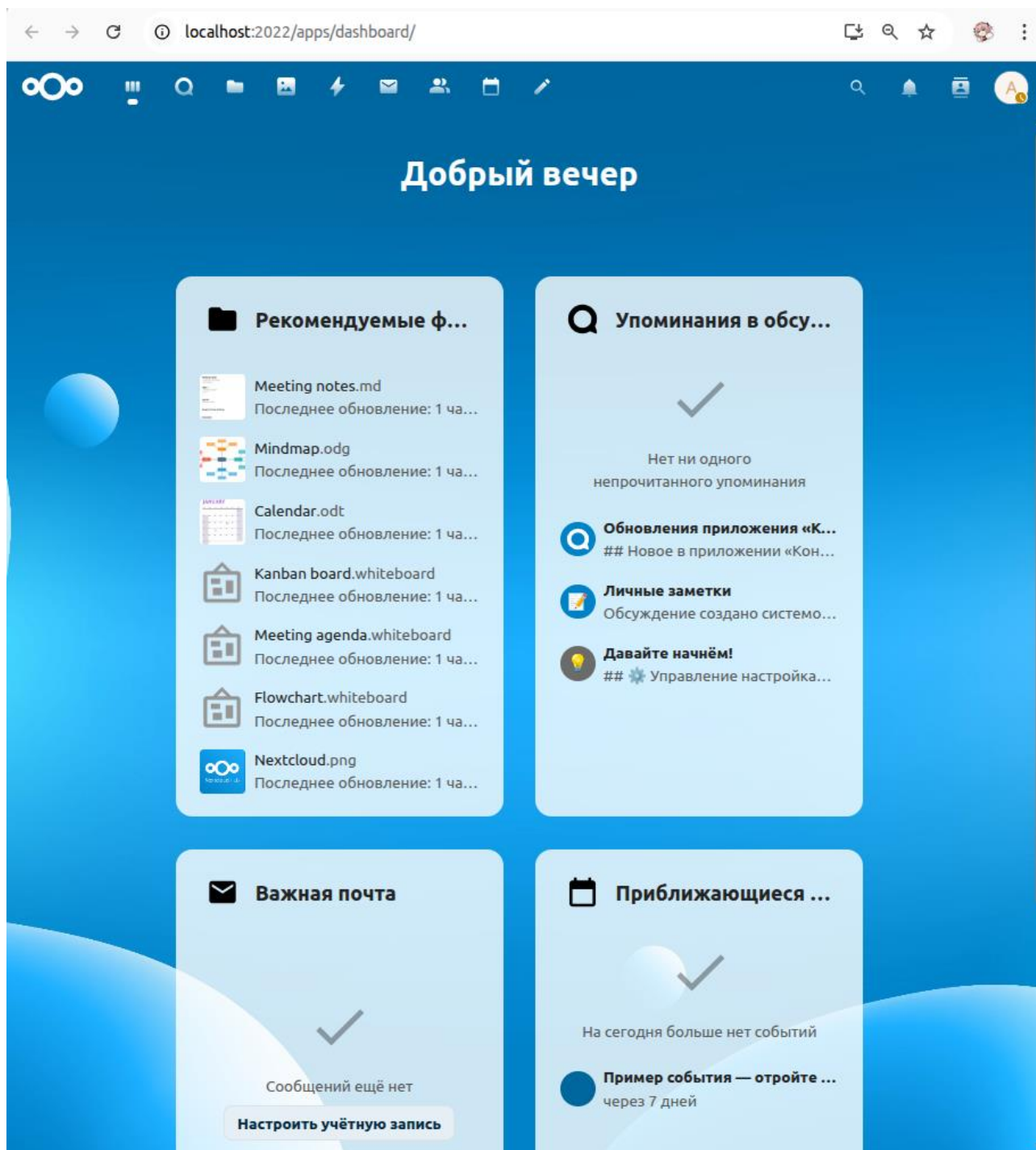


Рисунок 24. NextCloud через порт 2022

Смотрим пользователей, зарегистрированные ранее пользователи видны — рисунок 25.

Отображаемое имя	Имя учётной записи	Пароль	Адрес эл. почты	Участни
admin	admin			admin
Белозубов Александр ...	belozubov		belozubov@mail.ru	
Стрельбицкий Илья П...	strelbitskiy		strelbitskiy@mail.ru	

3 учётные записи

записями

Рисунок 25. Список пользователей

8. Регистрация на Docker Hub

Регистрируемся на <https://hub.docker.com> — что видно на рисунке 26.

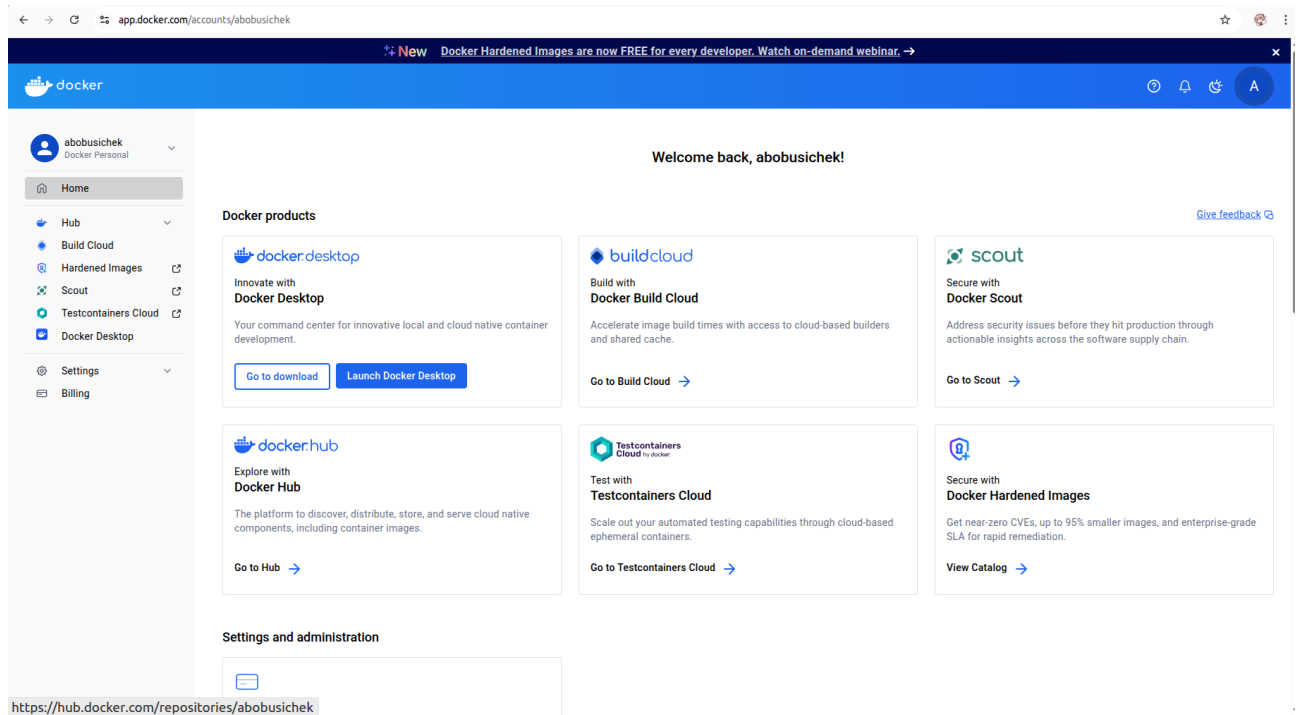


Рисунок 26. Мой профиль на docker.hub

9. Публикация образа на Docker Hub

Помечаем образ NextCloud тегом и отправляем в публичный репозиторий — рисунок 27.

```
sus@sus-computer:~/Desktop/university/tv/src$ docker tag nextcloud abobusichек/nextcloud:1.0
sus@sus-computer:~/Desktop/university/tv/src$ docker push abobusichек/nextcloud:1.0
The push refers to repository [docker.io/abobusichек/nextcloud]
be81e55c00c5: Mounted from library/nextcloud
d24a5b5dad69: Mounted from library/nextcloud
a905634016d2: Mounted from library/nextcloud
866c48bd8cb2: Mounted from library/nextcloud
a617e2324311: Mounted from library/nextcloud
a479f264b544: Mounted from library/nextcloud
95d3e67c4648: Mounted from library/nextcloud
c0ca8aff2e48: Mounted from library/nextcloud
5f70bf18a086: Mounted from library/nextcloud
d38700151fef: Mounted from library/nextcloud
5b6c99cf3a32: Mounted from library/nextcloud
69b655ac2fb7: Mounted from library/nextcloud
96745abf5a51: Mounted from library/nextcloud
3a9c8e3bc9b5: Mounted from library/nextcloud
bed4a85e4deb: Mounted from library/nextcloud
7aaa9f853e8f: Mounted from library/nextcloud
fe065384509c: Mounted from library/nextcloud
68bcf12be499: Mounted from library/nextcloud
de80168824f2: Mounted from library/nextcloud
33e4a36e1003: Mounted from library/nextcloud
698f82a2c2bf: Mounted from library/nextcloud
6183b7266d0c: Mounted from library/nextcloud
e50a58335e13: Mounted from library/nextcloud
1.0: digest: sha256:ec43c39dc0d40ceb5e4259340a6e3ba2d74ef2318f0247dabdbed5b7a719742e size:
5122
sus@sus-computer:~/Desktop/university/tv/src$
```

Рисунок 27. Публикация образа на docker hub

Опубликованный образ в публичном репозитории Docker Hub — рисунок 28.

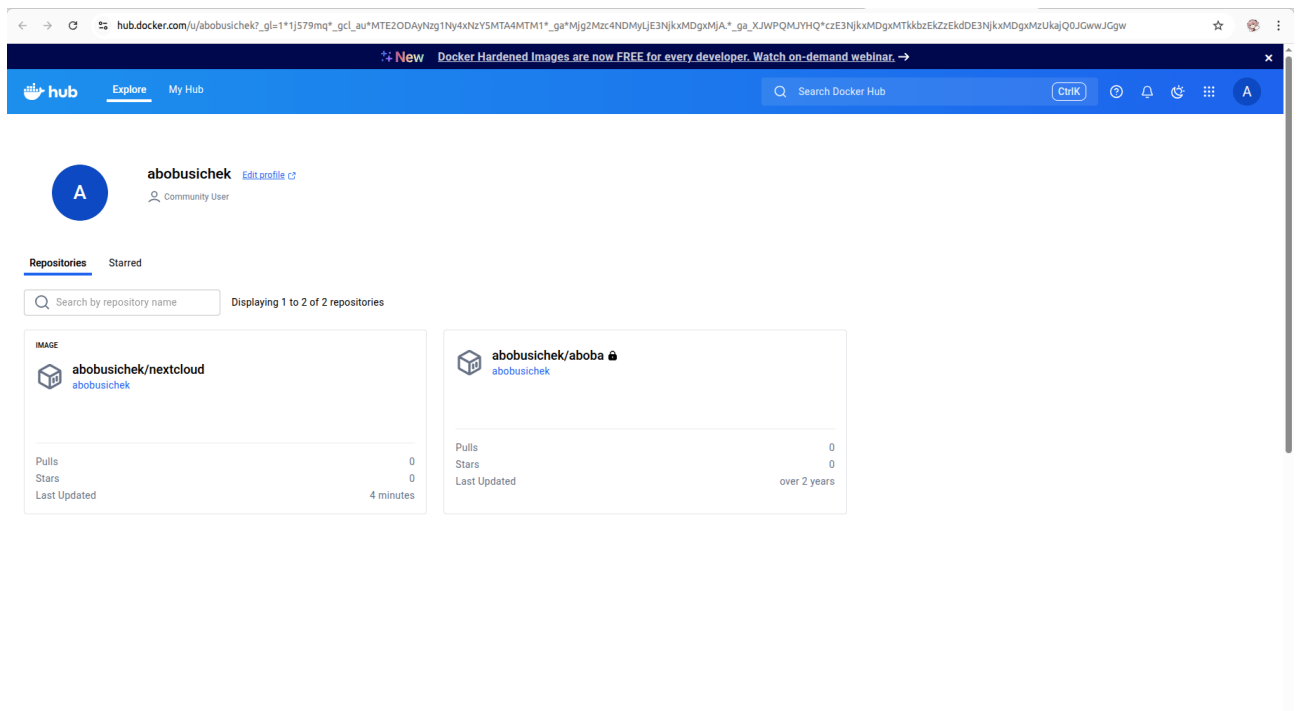


Рисунок 28. Опубликованный образ на docker hub

10. Запуск приложения из Docker Hub

Скачиваем и запускаем игру в терминале minesweeper — рисунок 29.

```
sus@sus-computer:~/Desktop/university/tv/src$ docker pull ghcr.io/bhb603/minesweeper-cli
Using default tag: latest
latest: Pulling from bhb603/minesweeper-cli
36576857ef94: Pull complete
Digest: sha256:9899c3365c4a4b3377e68c04b924a63faeb83e64194ad6a0d164e4b2a535ee29
Status: Downloaded newer image for ghcr.io/bhb603/minesweeper-cli:latest
ghcr.io/bhb603/minesweeper-cli:latest
sus@sus-computer:~/Desktop/university/tv/src$
docker run -it ghcr.io/bhb603/minesweeper-cli
Minesweeper [0.1.1]

8x8, 10 mines, 1 flagged

  0  0  0  0  0  0  2
  1  1  0  1  1  1  2
    1  0  1  1  1  1
    1  1  2  3  2  1
          [💣]

You lost. (Press any key to continue)

↑/k move up      enter reveal cell; reveal adjacent if all adj mines flagged
↓/j move down    f      flag/unflag
←/h move left    q      quit
→/l move right   ?      toggle help
```

Рисунок 29. Скачивание и запуск игры minesweeper

11. Основные команды Docker

В ходе выполнения лабораторной работы были применены следующие команды Docker для управления контейнерами, образами и многоконтейнерными приложениями.

Команды для управления контейнерами:

- `docker create` — создаёт контейнер из образа без его запуска
- `docker start` — запускает ранее созданный или остановленный контейнер
- `docker run` — создаёт и сразу запускает новый контейнер из образа
- `docker stop` — останавливает контейнер, отправляя сигнал завершения (SIGTERM)
- `docker kill` — принудительно завершает работу контейнера, отправляя сигнал SIGKILL
- `docker rm` — удаляет остановленный контейнер из системы
- `docker ps` — отображает список только запущенных контейнеров
- `docker ps -a` — отображает все контейнеры, включая остановленные
- `docker inspect` — выводит подробную информацию о контейнере
- `docker logs` — показывает журнал (логи) работы контейнера
- `docker exec` — выполняет указанную команду внутри уже запущенного контейнера
- `docker system df` — отображает использование дискового пространства Docker
- `docker volume ls` — перечисляет все тома, созданные Docker
- `docker network ls` — перечисляет все сети, управляемые Docker
- `docker version` — показывает версии Docker Engine и других компонентов
- `docker info` — выводит подробную информацию о конфигурации и состоянии Docker-окружения

Команды для управления образами:

- `docker build` — собирает образ на основе инструкций из файла Dockerfile
- `docker pull` — загружает образ из реестра
- `docker push` — отправляет локальный образ в удалённый реестр

- `docker images` (или `docker image ls`) — отображает список всех локальных образов
- `docker image inspect` — выводит детальную информацию о метаданных и конфигурации образа
- `docker rmi` (или `docker image rm`) — удаляет указанный образ из локального хранилища
- `docker login` — выполняет аутентификацию в Docker Hub или другом реестре контейнеров

Команды Docker Compose:

- `docker-compose up` — создаёт и запускает все сервисы, описанные в файле `docker-compose.yml`
- `docker-compose down` — останавливает и удаляет контейнеры, сети и тома, созданные через `docker-compose up`
- `docker-compose ps` — отображает статус всех сервисов в проекте
- `docker-compose logs` — выводит журналы всех запущенных сервисов
- `docker-compose build` — пересобирает образы, указанные в конфигурации Compose

Заключение

В ходе лабораторной работы был освоен полный цикл работы с Docker — от базовых операций до оркестрации многокомпонентных систем. Практика включала создание Dockerfile, сборку образов и развёртывание веб-сервера Nginx с пользовательской страницей.

Основной акцент был сделан на построении связанных приложений с помощью Docker Compose: успешно развёрнут стек из Nginx, СУБД MariaDB и облачного хранилища Nextcloud с корректной настройкой сети, переменных окружения и томов данных.

Дополнительно изучено управление виртуализацией через веб-интерфейс phpVirtualBox. Завершающим этапом стала интеграция с экосистемой Docker — публикация собственного образа в реестре Docker Hub.

Таким образом, были отработаны ключевые навыки работы с Docker: создание образов, оркестрация сервисов, настройка сетей и хранилищ, что формирует основу для работы с современными DevOps-инструментами.